

HEALTHCARE DATA PRIVACY ANALYZER

Privacy-Preserving Release of Healthcare Microdata

Complete Project Development Report

Prepared by

Atharva Kulkarni

B.Tech Computer Science & Engineering

G M University, Davangere

Technology

Python | Pandas | Streamlit | CSV | Git | GitHub

This report documents the project from environment setup and dataset creation through module development, Streamlit integration, debugging, testing, export, and GitHub publication.

1. Project Overview

The Healthcare Data Privacy Analyzer is an interactive Python application developed to demonstrate privacy-preserving release of healthcare microdata. It analyzes a small fictional healthcare dataset, classifies its attributes, calculates re-identification risk, applies privacy-preserving masking techniques, measures information loss, compares original and masked data, and exports the protected dataset.

The implementation follows the supplied Data Privacy assignment scope: a small fictional dataset, classification into Direct Identifier, Quasi-Identifier and Sensitive Attribute, re-identification risk calculation, at least two non-perturbative techniques, at least one perturbative technique, and privacy-utility analysis.

The final software turns the handwritten/theoretical exercise into a working Streamlit project.

2. Assignment Scope

Requirement	Implementation
Dataset	5 fictional healthcare records
Direct identifier	Patient ID
Quasi-identifiers	Age, Gender, PIN Code, Department
Sensitive attributes	Disease, Treatment
Risk formula	$1 / \text{number of records with same quasi-identifier combination}$
Non-perturbative masking	Suppression and Generalization
Perturbative masking	Data Swapping
Utility analysis	Information loss and changed cells

3. Project Objectives

No.	Objective
1	To design a privacy analysis system for identifying direct, quasi, and sensitive attributes in healthcare microdata.
2	To develop masking and re-identification risk analysis modules using Python and Pandas.
3	To implement an interactive Streamlit application for privacy protection, information-loss analysis, comparison, and protected-data exp

4. Tools and Technologies

Technology	Purpose
Python	Core programming language
Pandas	Dataset processing, grouping and comparison
Streamlit	Interactive web interface
CSV	Dataset input and protected output
VS Code	Development and editing
PowerShell	Environment setup and execution
Git	Version control
GitHub	Remote repository and publication

5. Project Setup - Step by Step

The project was created on Windows using PowerShell and VS Code.

```
cd D:\
mkdir Healthcare-Data-Privacy-Analyzer
cd Healthcare-Data-Privacy-Analyzer
code .
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install pandas streamlit matplotlib
pip freeze > requirements.txt
```

The virtual environment isolates project dependencies from the system Python installation.

6. Final Folder Structure

```
Healthcare-Data-Privacy-Analyzer/
|
+- app.py
+- requirements.txt
|
+- data/
|   +- healthcare_dataset.csv
|
+- modules/
    +- __init__.py
    +- classification.py
    +- risk_analysis.py
    +- masking.py
    +- utility_analysis.py
```

`__init__.py`: It is intentionally empty. Its role is to make modules a Python package so `app.py` can import the reusable functions.

7. Dataset Creation

The project uses the same five-record fictional healthcare dataset used for the assignment. No real personal information is used.

```
Patient ID, Age, Gender, PIN Code, Department, Disease, Treatment
P001, 22, F, 577001, Cardiology, Asthma, Inhaler
P002, 25, M, 577002, General, Diabetes, Medicine
P003, 31, F, 577001, Orthopedic, Arthritis, Therapy
P004, 45, M, 577004, Cardiology, Hypertension, Medicine
P005, 52, F, 577002, General, Diabetes, Medicine
```

The CSV was saved as data/healthcare_dataset.csv.

During development the application initially read the file as 5 x 1 instead of 5 x 7. The CSV was rewritten and verified with Pandas.

```
python -c "import pandas as pd; df=pd.read_csv('data/healthcare_dataset.csv'); print(df.shape); print(df.columns.tolist())"
(5, 7)
['Patient ID', 'Age', 'Gender', 'PIN Code', 'Department', 'Disease', 'Treatment']
```

8. Attribute Classification

Type	Attributes
Direct Identifier	Patient ID
Quasi-Identifier	Age, Gender, PIN Code, Department
Sensitive Attribute	Disease, Treatment

The classification module uses a dictionary containing the required categories and returns a DataFrame that is displayed by Streamlit.

```
classification = {  
    "Direct Identifier": ["Patient ID"],  
    "Quasi-Identifier": ["Age", "Gender", "PIN Code", "Department"],  
    "Sensitive Attribute": ["Disease", "Treatment"]  
}
```

9. Re-identification Risk Analysis

The risk module groups records by the four quasi-identifiers and calculates risk using the assignment formula.

$\text{Risk} = 1 / \text{Group Size} \times 100$

```
quasi_identifiers = [  
    "Age", "Gender", "PIN Code", "Department"  
]  
  
group_sizes = (  
    df.groupby(quasi_identifiers, dropna=False)  
        .size()  
        .reset_index(name="Group Size")  
)
```

For the original five records, every complete quasi-identifier combination is unique. Therefore group size is 1 for every record and average original risk is 100.00%.

10. Masking Techniques

Three techniques were implemented, satisfying the assignment requirement for at least two non-perturbative techniques and one perturbative technique.

10.1 Suppression

The exact PIN code is partially hidden by retaining the first three digits and replacing the remaining digits with asterisks.

```
577001 -> 577***  
577002 -> 577***  
577004 -> 577***
```

10.2 Generalization

Exact ages are converted to ranges, reducing precision while retaining age-group information.

```
22 -> 20-29  
25 -> 20-29  
31 -> 30-39  
45 -> 40-49  
52 -> 50-59
```

10.3 Data Swapping

Selected age values are exchanged between records. The values remain in the dataset, but their association with individual records changes.

```
if len(masked) >= 4:  
    masked.loc[0, "Age"], masked.loc[1, "Age"] = (  
        masked.loc[1, "Age"], masked.loc[0, "Age"]  
    )  
    masked.loc[2, "Age"], masked.loc[3, "Age"] = (  
        masked.loc[3, "Age"], masked.loc[2, "Age"]  
    )
```

11. Utility and Information-Loss Analysis

The utility module compares every cell in the original and masked datasets. The project contains $5 \times 7 = 35$ cells.

```
total_cells = original_df.shape[0] * original_df.shape[1]

changed_cells = 0

for column in original_df.columns:
    if column in masked_df.columns:
        for original, masked in zip(
            original_df[column], masked_df[column]
        ):
            if str(original) != str(masked):
                changed_cells += 1

information_loss = (
    changed_cells / total_cells
) * 100
```

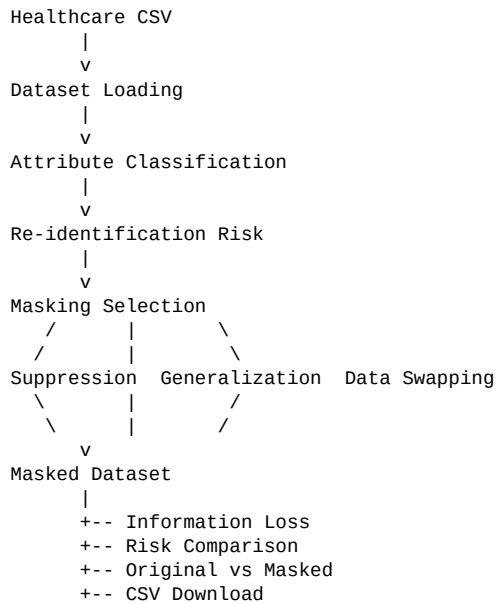
The dashboard reports both the percentage of information loss and the number of changed cells.

12. Streamlit Application

app.py imports all reusable functions and provides the interactive interface. The sidebar contains the masking selector: None, Suppression, Generalization, Data Swapping, and Apply All Non-Perturbative.

Section	Purpose
1. Dataset Overview	Records, attributes, dataset display
2. Attribute Classification	Direct, quasi, sensitive classification
3. Re-identification Risk Analysis	Group size and risk
4. Privacy-Preserving Masking	Selected technique and before/after data
5. Information Loss Analysis	Changed cells and information loss
6. Privacy Analysis	Original versus masked risk
7. Original vs Masked Dataset	Attribute-level comparison
8. Download Protected Dataset	Technique-specific CSV export

13. Main Application Workflow



The application therefore connects the assignment concepts into one continuous workflow.

14. Debugging and Fixes

CSV parsing fix: The first run displayed 5 x 1 instead of 5 x 7. The CSV was recreated in correct comma-separated format and verified using Pandas. After verification, all seven columns loaded correctly.

Masking display fix: The application was updated to show the selected technique, the original dataset, the masked dataset, and a table of changed attributes so the transformation is visible.

Download fix: Initially the same generic download control appeared to return the same dataset. The final implementation uses technique-specific filenames and Streamlit keys.

15. Final Download Implementation

```
filename_map = {
    "None": "original_healthcare_dataset.csv",
    "Suppression": "suppressed_healthcare_dataset.csv",
    "Generalization": "generalized_healthcare_dataset.csv",
    "Data Swapping": "swapped_healthcare_dataset.csv",
    "Apply All Non-Perturbative":
        "masked_healthcare_dataset.csv"
}

download_filename = filename_map[masking_option]

csv_data = masked_df.to_csv(index=False).encode("utf-8")

st.download_button(
    label=f"Download {masking_option} Dataset",
    data=csv_data,
    file_name=download_filename,
    mime="text/csv",
    key=f"download_{masking_option}"
)
```

This guarantees that the exported filename identifies the selected technique and that the current masked DataFrame is exported.

16. Testing

Test	Expected result	Status
None	Original dataset unchanged	Working
Suppression	PIN becomes 577***	Working
Generalization	Age becomes ranges	Working
Data Swapping	Selected age values exchange records	Working
Apply All Non-Perturbative	Suppression + generalization	Working
Download	Current selected result is exported	Working

17. Important Result Interpretation

The original dataset has 100.00% average risk because every complete quasi-identifier combination is unique. After masking, the displayed risk can remain 100% if the transformed quasi-identifier combinations are still unique.

This does not mean the masking operation failed. The application separately displays the actual transformation and information loss. The risk value is reported according to the assignment's specified formula rather than being artificially reduced.

For example, generalization changes 22 to 20-29 and 31 to 30-39, which reduces precision even if the resulting combinations remain unique in this very small five-record dataset.

18. Final Project Structure

```
Healthcare-Data-Privacy-Analyzer/  
|  
+- app.py  
+- requirements.txt  
|  
+- data/  
|   +- healthcare_dataset.csv  
|  
+- modules/  
    +- __init__.py  
    +- classification.py  
    +- risk_analysis.py  
    +- masking.py  
    +- utility_analysis.py
```

19. GitHub Publication

The completed project was prepared for the GitHub repository Healthcare-Data-Privacy-Analyzer. The repository contains the application, dataset, modules, requirements file, and README.

The virtual environment should not be committed. A .gitignore file can contain:

```
venv/  
__pycache__/  
*.pyc  
.streamlit/
```

Final publication commands:

```
git add .  
git commit -m "Complete Healthcare Data Privacy Analyzer"  
git push origin master
```

The README documents the project overview, features, technology stack, structure, privacy techniques, risk formula, setup instructions, export files, academic context, and author.

20. Results

Area	Result
Dataset	5 fictional records, 7 attributes
Classification	All attributes categorized
Original risk	100.00% average
Suppression	PIN precision reduced
Generalization	Age precision reduced to ranges
Data swapping	Selected age values reassigned
Information loss	Calculated from changed cells
Comparison	Original and masked data displayed
Export	Technique-specific CSV files

21. Conclusion

The Healthcare Data Privacy Analyzer was successfully developed as a complete working privacy-analysis application. Starting from a small fictional healthcare dataset, the project implemented attribute classification, re-identification risk calculation, suppression, generalization, data swapping, information-loss analysis, dataset comparison, and protected-data export.

The development process also included practical debugging: correcting CSV parsing, verifying the dataset shape and columns, making masking transformations visible, and fixing the download feature so that the exported file corresponds to the selected technique.

The project demonstrates the privacy-utility trade-off in a practical way. Masking can reduce the precision of data and therefore introduce information loss, while the exact risk outcome depends on the quasi-identifier combinations defined by the risk model.

The final application provides a strong foundation for future extensions such as larger datasets, configurable quasi-identifiers, additional masking algorithms, visual risk charts, k-anonymity, l-diversity, and more advanced privacy models.

References

1. Faculty assignment instructions: Privacy-Preserving Release of Microdata: Re-identification Risk and Data Masking.
2. Project implementation developed using Python, Pandas and Streamlit.
3. Git and GitHub used for version control and project publication.

End of Report